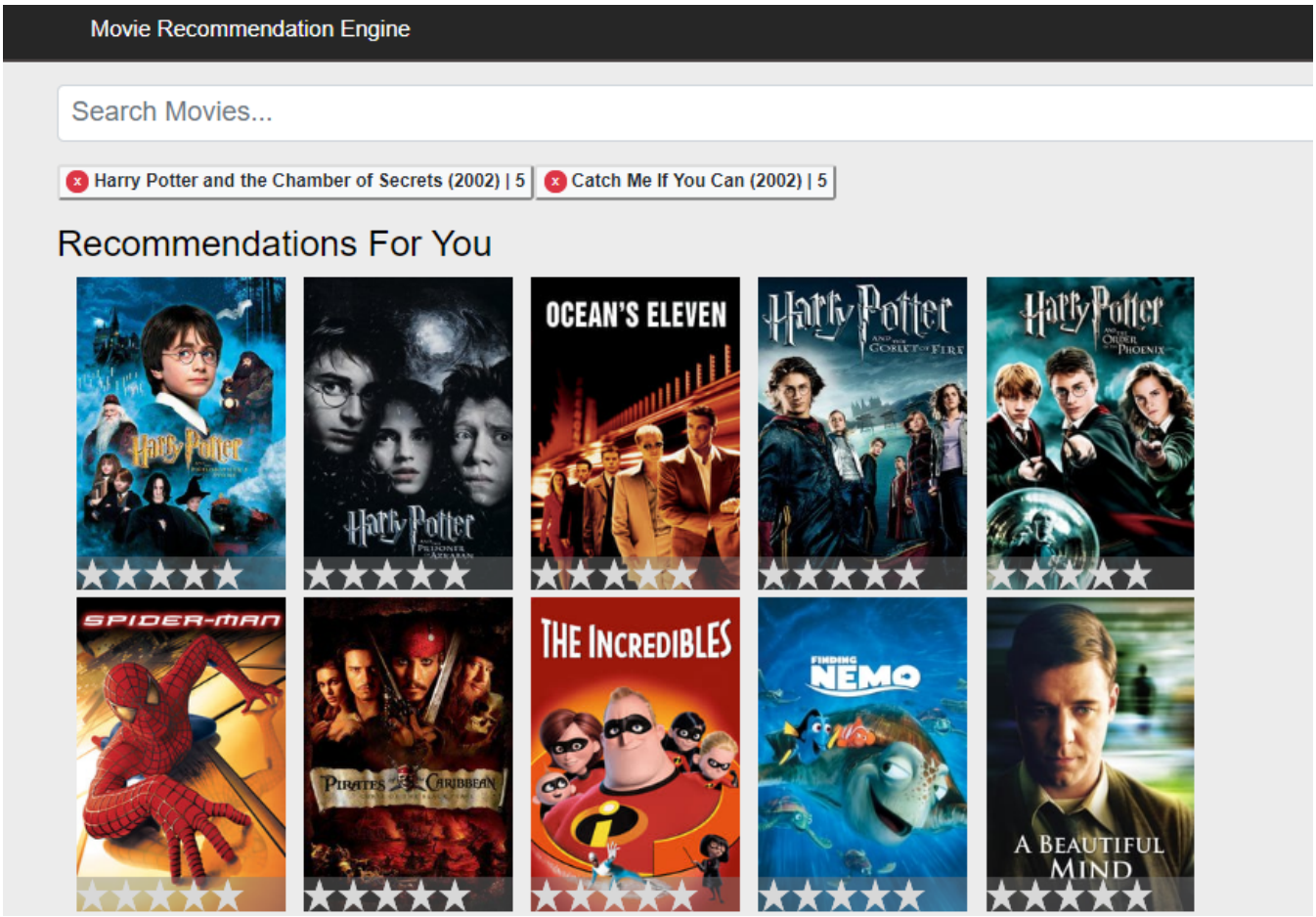
How to Turn Your Machine Learning Scripts into Projects You Can Demo?

Code Heroku
Sep 9, 2019 · 6 min read



Recently we published an article and video on Building a Movie Recommendation Engine and one of the most common questions that I received was — “How do I turn this script/notebook into a project that I can demo?”. So, I decided to build a front-end for our recommendation engine. You can try it out here:

> *http://www.codeheroku.com/static/movies/index.html*

Movie Recommendation System with Collaborat

Movie Recommendation Engine with Collaborative Filtering

In our previous videos we have covered building a recommendation system using Collaborative Filtering. In this post, I would like to start from a point assuming that you have built a recommendation model and now want to create a web based user interface for it.

```
##Ingredients For this Recipe:

1. Web2py Framework : Download Here
2. Recommendations Model File:  item_similarity_df.csv
3. Test HTML Page: Download Here
4. Completed Application Code with all above: Download Here
```

Let's go through high-level steps for building a user interface for your ML application:

- Selection of Tools and Languages
- Building APIs/Services to expose your ML model
- Building Client-side Web / Mobile App

Step 1: Selection of Tools and Programming Languages

Whether you are a veteran software developer or somebody who is just starting out, this is a crucial step that will determine how much time and efforts you will need to build the application you envisioned.

You must use the right tools for the right purpose!

Assuming you have built your ML model in Python, it makes sense to use this powerful language for your web applications/API engine as well. It also means that you don't have to port any of your code to any other programming language. Here's my recommendation on a stack that you can't go wrong with:

> *Build APIs using a Python based framework (Flask, web2py, Django) and*
>
> *Build your Front-end using JavaScript with VueJS or ReactJS*

If you love Python, you would like JavaScript. Using reactive packages like VueJS/ReactJS is optional but will make your life simpler especially to build a single page app as you won't have to worry about refreshing pages and passing parameters from one page to another!

In this post we will be using a Python based web framework called web2py for building APIs and VueJS for building reactive HTML front-end.

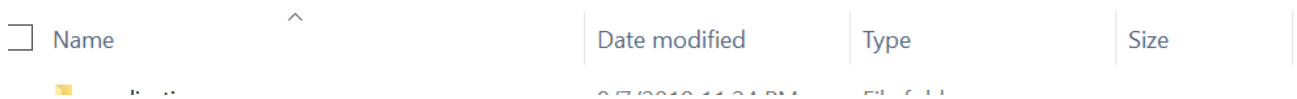
Step 2: Building APIs/Services to expose your ML model

Our Machine Learning models will be exposed to outside world using a web service. We can then consume this service in a mobile / web application to make our apps smarter. We will create a service called getRecommendations

```
@service.json
def getRecommendations(watched_movies):

     ##Magic with our ML model goes here

return recommended_movies
```

This is a JSON API that takes in movies that a user has watched/rated in the past and returns a list of recommended movies. web2py is great for building APIs with minimal lines of code. So let's go ahead and Download web2py from here.

Once you have extracted the archive, you should see the following directory structure:



applications	9/7/2019 11:24 PM	File folder	
examples	9/7/2019 11:23 PM	File folder	
extras	9/7/2019 11:23 PM	File folder	
gluon	9/7/2019 11:23 PM	File folder	
handlers	9/7/2019 11:23 PM	File folder	
scripts	9/7/2019 11:23 PM	File folder	
site-packages	9/7/2019 11:24 PM	File folder	
anyserver.py	9/7/2019 11:23 PM	Python File	13 KB
CHANGELOG	9/7/2019 11:23 PM	File	66 KB
fabfile.py	9/7/2019 11:23 PM	Python File	6 KB
LICENSE	9/7/2019 11:23 PM	File	6 KB
MANIFEST.in	9/7/2019 11:23 PM	IN File	1 KB
NEWINSTALL	9/7/2019 11:23 PM	File	1 KB
README.markdown	9/7/2019 11:23 PM	MARKDOWN File	4 KB
VERSION	9/7/2019 11:23 PM	File	1 KB
web2py.py	9/7/2019 11:23 PM	Python File	1 KB

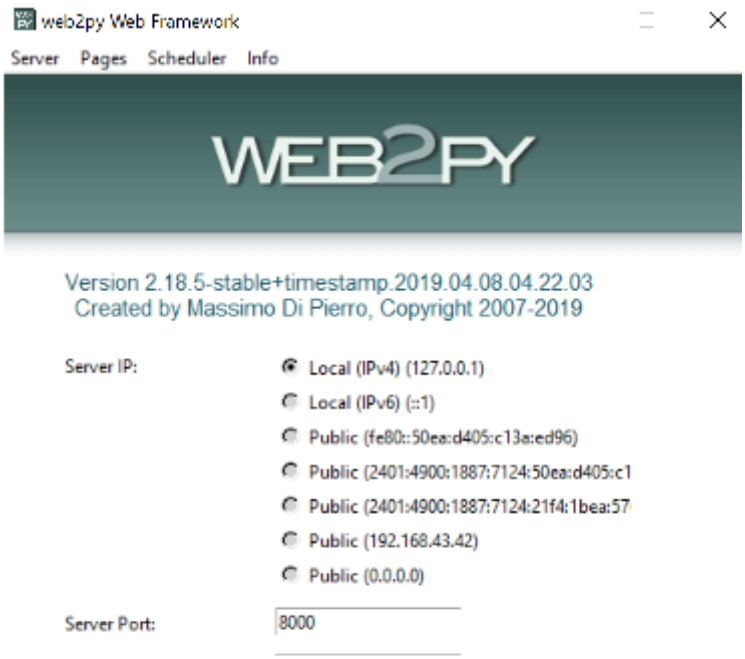
web2py directory structure

A quick note on some important folders / files:

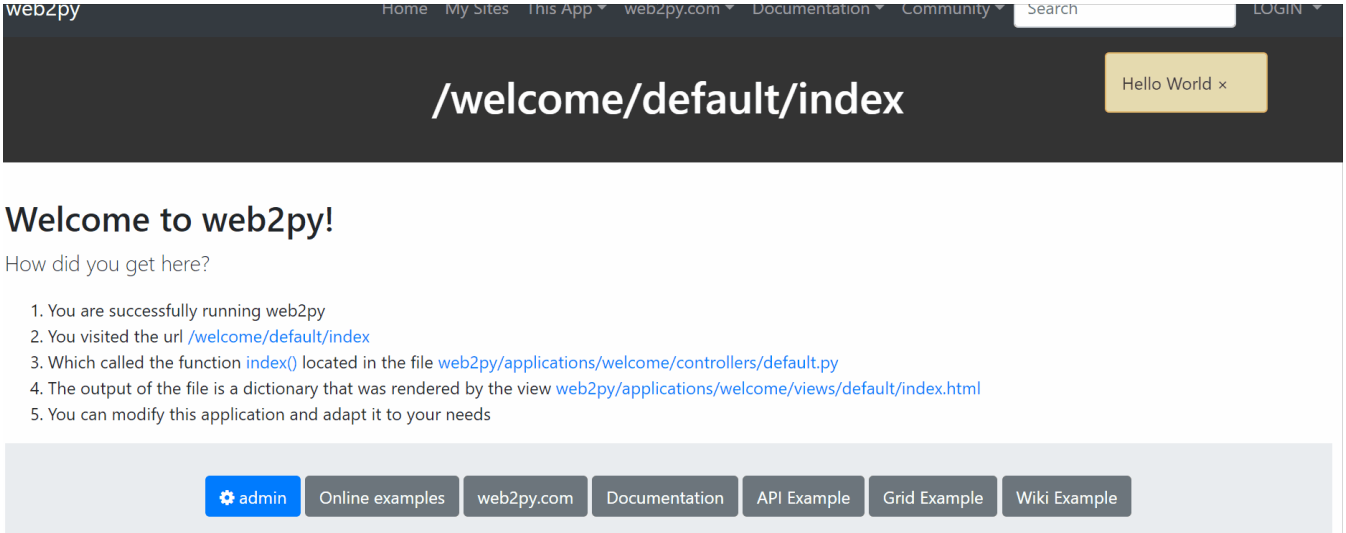
1. Applications: All your web2py apps/projects are contained in this directory. You can backup your projects and install new ones by simply copy-pasting your project folder inside this directory.

2. web2py.py: We can start our web server by running this script:

```
>> cd Downloads/web2py_src
>> python web2py.py
```

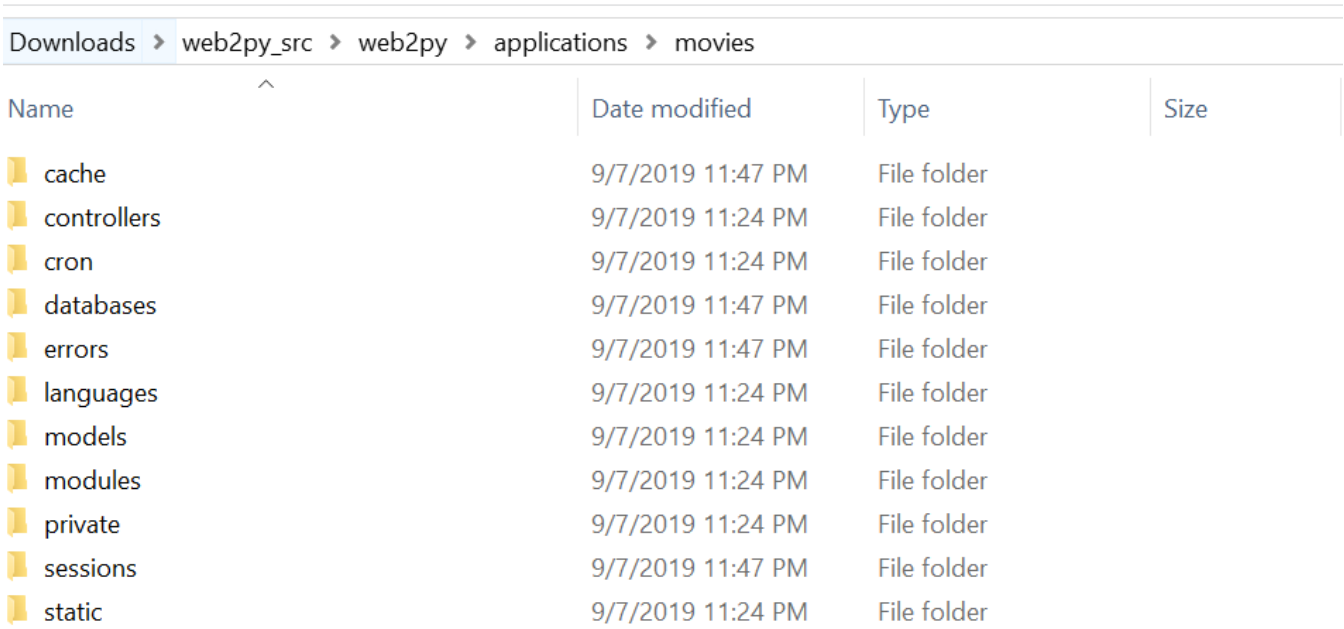


Choose a password of your choice and hit start server. Next, go to admin, use the password you just used and then create a simple application called “movies”



Now, if we want we could write the rest of the code in the buit-in web based editor that web2py provides. But I am not a huge fan of this editor! So, let’s go back to the web2py directory and edit the app in our favorite code editor— Sublime/Atom/VS Code.

Head to the applications folder and you should see a new folder created for our movies app. Here’s the directory structure of our web2py movies app:



uploads	9/7/2019 11:47 PM	File folder	
views	9/7/2019 11:24 PM	File folder	
__init__.py	9/7/2019 11:24 PM	Python File	1 KB
ABOUT	9/7/2019 11:24 PM	File	1 KB
LICENSE	9/7/2019 11:24 PM	File	1 KB
progress.log	9/7/2019 11:47 PM	Text Document	1 KB
routes.example.py	9/7/2019 11:24 PM	Python File	3 KB

Web2py inherently supports MVC (Model View Controller) architecture.

- **M**odels: define database schema and validations
- **V**iews: hosts your HTML code
- **C**ontrollers: Python code where you will write business logic and manages interactions between front-end and the database.

Even though MVC is a good architectural pattern, the world has moved on to using micro-services architecture so that our front-end is independent of the controller logic (mostly). Right now, we will be focusing only on the controllers where we will build our APIs. Open the controller file located at applications/movies/controllers/default.py

First, let me show you how easy it is to use web2py and create a simple API. let's create an API called *myapi* that takes in two parameters a & b and returns a string concatenation of these two parameters:

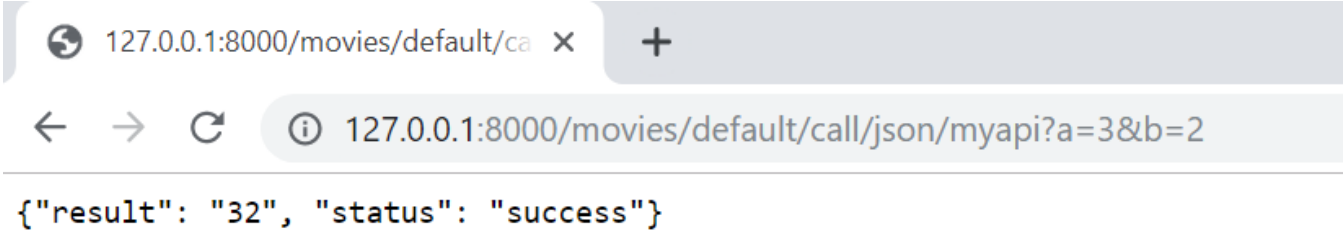
```
from gluon.tools import Service
service = Service()

def call():
    return service()

@service.json
def myapi(a,b):
    print("got: ",a,b)
    #concat two strings
    result = a+b
    return dict(result=result,status="success")
```

default.py hosted with ❤ by **GitHub** **view raw**

Create a Simple API in applications/movies/controllers/default.py

Ok, don't worry yet — In lines 1 through 5 we wrote some web2py specific code which is used to initialize web services in our app. The @service.json decorator tells web2py that we want to expose myapi as a JSON web service. You can also try @service.xml or @service.csv to build services in other protocols.

Now, let's call our API from the browser and see if it actually works. Go to the following URL in your browser: http://127.0.0.1:8000/movies/default/call/json/myapi?a=3&b=2



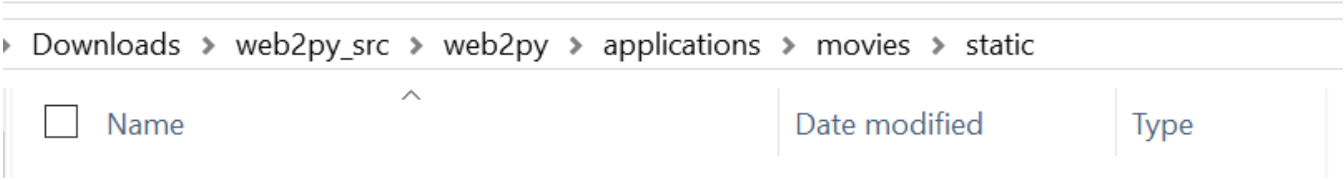
Testing our simple API by passing two parameters and getting a concat of two strings in the result.

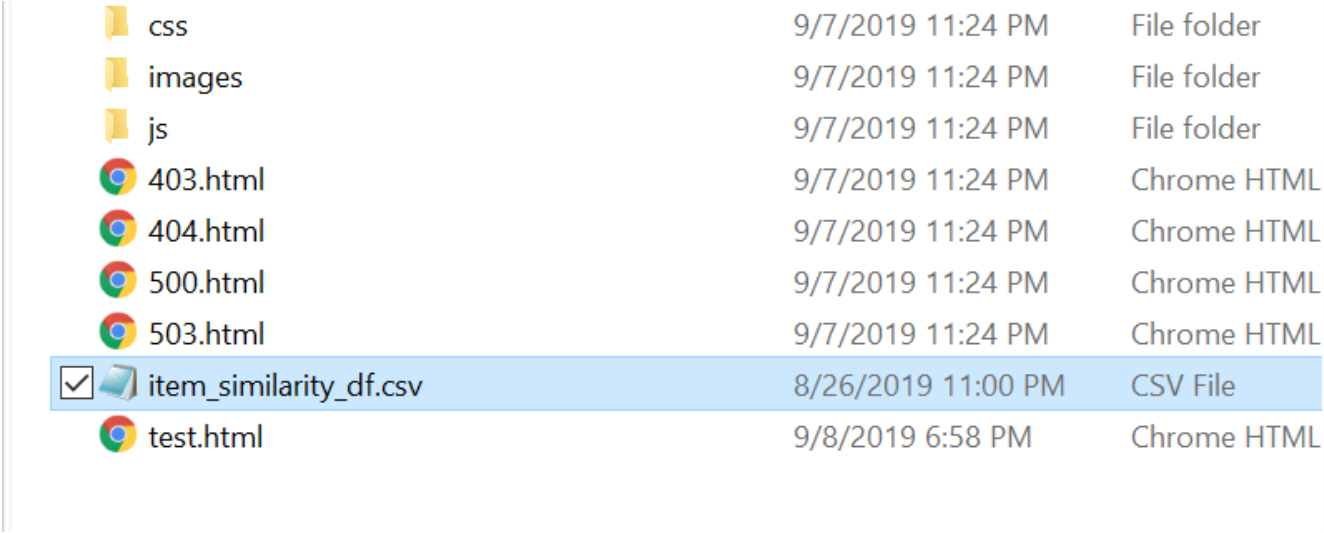
Congratulations! you have just created your first API!

Now its time to build our Recommendations API. Recall our API signature look something like this:

```
@service.json
def getRecommendations(watched_movies):

     ##Magic with our ML model goes here

return recommended_movies
```

In the first step, we will load the recommendations model that we built using Collaborative Filtering in our app. We have created an "item_similarity_df" in our collaborative filtering tutorial. Click Here to download this CSV file and place it in your applications/movies/static folder.





css	9/7/2019 11:24 PM	File folder
images	9/7/2019 11:24 PM	File folder
js	9/7/2019 11:24 PM	File folder
403.html	9/7/2019 11:24 PM	Chrome HTML
404.html	9/7/2019 11:24 PM	Chrome HTML
500.html	9/7/2019 11:24 PM	Chrome HTML
503.html	9/7/2019 11:24 PM	Chrome HTML
item_similarity_df.csv	8/26/2019 11:00 PM	CSV File
test.html	9/8/2019 6:58 PM	Chrome HTML

Since we do not want to load the data frame every time someone uses our app we will cache it using web2py's cache.ram method.

```
import pandas as pd
def load_recommendations():
    item_similarity_df = pd.read_csv("applications/movies/static/item_similarity_df.csv'
    print("item_similarity_df cached in memory")
    return item_similarity_df

item_similarity_df = cache.ram('item_similarity_df3',load_recommendations,None)
print(item_similarity_df.head())
```

default.py hosted with ❤ by **GitHub** **view raw**

Next up, comes the part we were waiting for: Building the recommendation API. This is pretty much the same method we created during collaborative filtering class with minute changes on the data type of watched_movies. We have also included a helper method that excludes movies that user has already watched/rated from the list of recommendations.

```
##helper method that doesn't recommend a movie if the user has already seen it
def check_seen(recommended_movie,watched_movies):
    for movie_id,movie in watched_movies.items():
        if recommended_movie == movie["title"]:
            return True
    return False

def get_similar_movies(movie_name,user_rating):
    try:
        similar_score = item_similarity_df[movie_name]*(user_rating-2.5)
        similar_movies = similar_score.sort_values(ascending=False)
```

```
    except:
        print("don't have movie in model")
        similar_movies = pd.Series([])

    return similar_movies

@service.json
def get_recommendations(watched_movies):

    print(watched_movies)
    similar_movies = pd.DataFrame()

    for movie_id,movie in watched_movies.items():
        similar_movies = similar_movies.append(get_similar_movies(movie["title"],movie

    all_recommend = similar_movies.sum().sort_values(ascending=False)

    recommended_movies = []
    for movie,score in all_recommend.iteritems():
        if not check_seen(movie,watched_movies):
            recommended_movies.append(movie)

    if len(recommended_movies) > 100:
        recommended_movies = recommended_movies[0:100]

    return recommended_movies
```

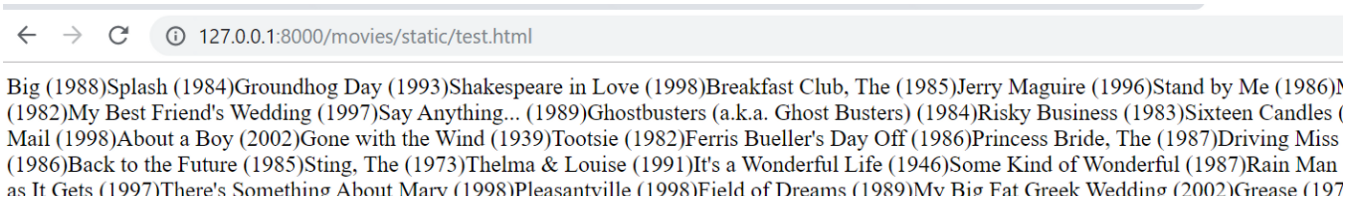
default.py hosted with ❤ by **GitHub** **view raw**

Testing Your Recommendations API

To test our API we need to send a POST request to the API located at /movies/default/call/json/get_recommendations with watched_movies parameter.

To make things easier for you, we have created a test HTML page that sends this request. Go ahead and download it from here and save this page in your applications/movies/static folder. Right click > Save page as (uhumm..uhumm)

You can access this page on your browser at URL http://127.0.0.1:8000/movies/static/test.html



(2001)Romancing the Stone (1984)Being There (1979)Raising Arizona (1987)Desperately Seeking Susan (1985)Killing Fields, The (1984)Wizarc
News (1987)Blues Brothers, The (1980)To Catch a Thief (1955)On Golden Pond (1981)Monty Python and the Holy Grail (1975)Annie Hall (197
(1983)Dead Poets Society (1989)Pretty Woman (1990)Hunt for Red October, The (1990)Who Framed Roger Rabbit? (1988)Big Chill, The (1983)
(1963)Four Weddings and a Funeral (1994)Sabrina (1954)Christmas Story, A (1983)Heathers (1989)Little Mermaid, The (1989)Full Monty, The (
It Hot (1959)Good Morning, Vietnam (1987)Witness (1985)That Thing You Do! (1996)Grifters, The (1990)Indiana Jones and the Temple of Door
Singer, The (1998)Back to the Future Part III (1990)Back to the Future Part II (1989)L.A. Story (1991)Dirty Dancing (1987)Grosse Pointe Blank
(1960)High Fidelity (2000)Notorious (1946)Being John Malkovich (1999)Heaven Can Wait (1978)Bull Durham (1988)Lethal Weapon (1987)Ani
Men Don't Wear Plaid (1982)League of Their Own, A (1992)Fifth Element, The (1997)This Is Spinal Tap (1984)Almost Famous (2000)Contact (1

This test.html page sends a post request with a sample movie the user has rated and returns recommended movies for the user.

If you see a page like this that means your API is working as expected and you are all set for the next part of this tutorial.

In the next part we will build the HTML/JavaScript based front-end using VueJS to call this API and display recommended movies for our users in a nice looking interface. We will decide to publish the next part based on the feedback and number of responses we get for this post. So if you are keen on getting the next part, please do comment on this post or reach out to us on WhatsApp.

```
##Ingredients For this Recipe:

1. Web2py Framework : Download Here
2. Recommendations Model File:  item_similarity_df.csv
3. Test HTML Page: Download Here
4. Completed Application Code with all above: Download Here
```

> *If you like this post, please do share it with your network and give it some claps! This will encourage us to produce more such content.*
>
> *If you get stuck somewhere while building this project you can reach out to us via email: hello@codeheroku.com or on WhatsApp: +91–9967578720*

Machine Learning Projects Python Recommendation System Recommendation Engine Machine Learning

About Help Legal